

CSE3150

Lab Assignment: Building a Generic Vector

Dr. Justin Furuness

Refresher: Equality, Iterators, and Concepts

IMPORTANT: This section is just to refresh some of these ideas. You do NOT need to code anything at all for the refresher section.

Before we build our own `Vector`, let us review three connected ideas:

- checking whether two values are equal
- checking whether two containers are equal by looping through them with iterators
- using a C++20 concept to describe when two values are allowed to be compared

This is a useful refresher because later in the lab:

- our vector will store values of a generic type `T`
- our vector equality operator will compare those values one by one
- our iterator will let us walk through the container

Refresher Step 1: Compare two values

Let us start with the most basic version:

```
bool equals(int a, int b) {  
    return a == b;  
}
```

This works fine for integers, but only for integers.

We can make it generic with a template:

```
template <typename T>  
bool equals_basic(const T& a, const T& b) {  
    return a == b;  
}
```

Now this works for many types, as long as `T` supports `==`.

Refresher Step 2: Compare two containers by looping with iterators

Now let us write a separate function that compares two containers element by element.

This function will not worry about the details of how values are compared. It will just loop through the containers using iterators and call our helper function.

```
template <typename Container>
bool container_equals(const Container& a, const Container& b) {
    auto it1 = a.begin();
    auto it2 = b.begin();

    while (it1 != a.end() && it2 != b.end()) {
        if (!equals_basic(*it1, *it2)) {
            return false;
        }
        ++it1;
        ++it2;
    }

    return it1 == a.end() && it2 == b.end();
}
```

This is nice because the jobs are separated:

- `container_equals` handles looping through the container
- `equals_basic` handles comparing two values

So the iterator logic and the equality logic are not mixed together too much.

Refresher Step 3: Add a concept to the value-comparison helper

Now let us improve the generic equality helper using a C++20 concept.

The important requirement is simple: the two values must support `==`.

So we can define this concept:

```
#include <concepts>

template <typename T>
concept EqualityComparable = requires(const T& a, const T& b) {
    { a == b } -> std::convertible_to<bool>;
};
```

Now we apply the concept only to the value-comparison helper:

```
template <EqualityComparable T>
bool equals(const T& a, const T& b) {
    return a == b;
}
```

Notice how much simpler this is than trying to attach the concept to the whole container loop. The concept is describing exactly one small idea:

This function only works for types that can be compared with `==`.

Then our container function can just call that helper:

```
template <typename Container>
bool container_equals(const Container& a, const Container& b) {
    auto it1 = a.begin();
    auto it2 = b.begin();

    while (it1 != a.end() && it2 != b.end()) {
        if (!equals(*it1, *it2)) {
            return false;
        }
        ++it1;
        ++it2;
    }

    return it1 == a.end() && it2 == b.end();
}
```

Final refresher code

Here is the complete example with the final code:

```
#include <iostream>
#include <vector>
#include <string>
#include <concepts>

template <typename T>
concept EqualityComparable = requires(const T& a, const T& b) {
```

```

    { a == b } -> std::convertible_to<bool>;
};

template <EqualityComparable T>
bool equals(const T& a, const T& b) {
    return a == b;
}

template <typename Container>
bool container_equals(const Container& a, const Container& b) {
    auto it1 = a.begin();
    auto it2 = b.begin();

    while (it1 != a.end() && it2 != b.end()) {
        if (!equals(*it1, *it2)) {
            return false;
        }
        ++it1;
        ++it2;
    }

    return it1 == a.end() && it2 == b.end();
}

int main() {
    std::vector<int> a{1, 2, 3};
    std::vector<int> b{1, 2, 3};
    std::vector<int> c{1, 2, 4};

    std::cout << container_equals(a, b) << "\n";
    std::cout << container_equals(a, c) << "\n";

    return 0;
}

```

Overview

In this lab, you will build your own simplified generic **Vector** class in C++. This lab is designed to help reinforce several important ideas that come up often in modern C++:

- Templates for generic programming
- Writing generic classes, not just generic functions
- Dynamic memory management
- References and returning references from functions
- Operator overloading
- Const correctness
- Iterators and range-based for loops

Unlike a homework, this lab is meant to be more guided. In some places I will leave pieces for you to design and implement yourself. In other places, especially once we get to iterators, I will walk you through the code step by step.

Your final code should live in a file named:

`vector.hpp`

You may also create a small `main.cpp` file locally to test your implementation.

Goal

By the end of this lab, your `Vector` class should support:

- Generic types using templates
- `push_back`
- `size`, `capacity`, and `empty`
- Bounds-checked access through `at`
- Bracket access through `operator[]`
- Equality comparison through `operator==`
- Iteration with `begin()` and `end()`
- Range-based for loops

1 Phase 1 Build a Generic Vector Class

So far we have seen generics mostly with functions, but C++ templates can also be used with entire classes. That means instead of writing one vector for `int`, another for `double`, and another for `std::string`, we can write one generic class and let the compiler generate the needed version for us.

For example:

```
template <typename T>
class Vector {
    // ...
};
```

Here, `T` is the generic type parameter. If someone writes `Vector<int>`, then `T` becomes `int`. If someone writes `Vector<std::string>`, then `T` becomes `std::string`.

What your class should contain by the end of this phase

Your `Vector` class should have these private attributes:

- `T* data_;`
- `size_t size_;`
- `size_t capacity_;`

It should also have this private helper function:

- `resize_capacity`

And it should have these public members:

- A constructor
- A destructor
- `size()`
- `capacity()`
- `empty()`
- `push_back()`

Some design guidance

I want this phase to still be somewhat open ended, so I am not going to hand you all of the code. Instead, think through the following:

- What should the constructor initialize `data_`, `size_`, and `capacity_` to?
- When should `push_back()` call `resize_capacity()`?
- What should happen to the old array when the capacity grows?
- What value should the capacity become when you resize?

A common pattern is:

- Start with capacity 0 or 1
- When full, grow to 2x the old capacity
- Allocate a new array
- Copy values over
- Delete the old array

Suggested class skeleton

You should start from a skeleton like this and fill in the function bodies yourself for the following code:

```
#pragma once
#include <cstddef> // for size_t
#include <stdexcept>

template <typename T>
class Vector {
private:
    T* data_;
    size_t size_;
    size_t capacity_;

    void resize_capacity(size_t new_capacity);

public:
```

```

    Vector();
    ~Vector();

    size_t size() const;
    size_t capacity() const;
    bool empty() const;

    void push_back(const T& value);

    T& at(size_t index);
};

```

Returning references from functions

One important idea for this lab is that functions can return references.

The real `std::vector` does this with `at()` and `operator[]`. That means if you access an element and store it in a reference, changing that reference changes the original object in the vector.

For example:

```

#include <vector>
#include <iostream>

int main() {
    std::vector<int> nums{10, 20, 30};

    int& x = nums.at(1);
    x = 999;

    std::cout << nums.at(1) << "\n"; // prints 999
}

```

Why did that happen? Because `nums.at(1)` returned a reference to the actual element inside the vector, not just a copy.

This is a very useful pattern. It means your function return types can themselves be references:

```

T& at(size_t index);

```

Your task: implement at

You should now implement:

```
T& at(size_t index);
```

Requirements:

- If the index is invalid, throw an exception
- Otherwise return a reference to the element at that index

You should decide:

- Which exception type to throw
- What error message to use

Don't forget to implement all the other functions as well. Make sure there are no memory leaks!!

A quick note on copying

There is one small issue with our **Vector** class that we are not ready to fully solve yet.

Because this class manages dynamic memory internally using a raw pointer, C++ has special rules about what should happen when an object is copied. The language provides features called a **copy constructor** and a **copy assignment operator** to control that behavior.

We have not gone over copy constructors yet in class, so for this lab we are simply going to disable copying.

Do not worry yet about exactly what these two lines are doing. For now, just put them in your **Vector** class:

```
Vector(const Vector&) = delete;  
Vector& operator=(const Vector&) = delete;
```

This tells the compiler that **Vector** objects should not be copied for this lab.

So for now, add these lines to the public section of your class and move on. We will come back later in the course and explain exactly why this matters and how to properly support copying for classes that manage memory.

2 Phase 2 Operator Overloading and Const Correctness

C++ lets us overload operators so that our own classes can behave more like built-in types.

For example, if we want this to work:

```
Vector<int> v;  
v.push_back(5);  
v.push_back(10);  
  
std::cout << v[1] << "\n";
```

then we need to overload `operator[]`. That way our custom `Vector` class will also support this bracket access.

Why do we need two versions of operators?

We also need to think about `const` correctness.

If the vector is non-const, then bracket access should allow modification:

```
v[0] = 42;
```

But if the vector is const, then bracket access should not allow modification.

That means we want two overloads:

- one returning `T&`
- one returning `const T&`

Here are the stubs:

```
T& operator[](size_t index);  
  
const T& operator[](size_t index) const;
```

You should implement both of these now. To reiterate: Adding `const` actually changes the underlying function that gets called. Remember the function overloading that we went over in lectures. This applies to const return types as well!

Your next task: implement `operator==`

Now implement equality for vectors. We want to be able to write:

```
Vector<int> a;
Vector<int> b;

if (a == b) {
    // ...
}
```

Your `operator==` should return true only if:

- the vectors have the same size
- every corresponding element is equal

Write and implement:

```
bool operator==(const Vector<T>& other) const;
```

A new concept: C++20 Concepts and requires

There is something subtle happening with our generic `Vector`.

When we implement:

```
bool operator==(const Vector<T>& other) const;
```

we are assuming that the type `T` itself can be compared with `==`. That is because inside this function, we will eventually need code like:

```
data_[i] == other.data_[i]
```

If `T` does not support equality, then this code will not compile.

Before C++20, we would often just mention this requirement in comments or documentation and let template errors appear later during compilation. But in C++20, we can express this requirement directly in the code using a **concept**.

A **concept** lets us describe what operations a type must support in order to be used in a template.

For this lab, let us define a concept that says a type must support equality comparison.

```
#include <concepts>
```

```
template <typename T>  
concept EqualityComparable = requires(const T& a, const T& b) {  
    { a == b } -> std::convertible_to<bool>;  
};
```

Let us break this down:

- `concept EqualityComparable` defines a named requirement on types
- `requires(...)` says “for this type, the following expressions must be valid”
- `{ a == b } -> std::convertible_to<bool>;` says that `a == b` must compile, and the result must behave like a boolean

Now we can use this concept in our vector class:

```
template <EqualityComparable T>  
class Vector {  
    // ...  
};
```

This means the compiler will only allow types `T` that satisfy the `EqualityComparable` concept.

You can also write this using `requires` after the template line:

```
template <typename T>  
requires EqualityComparable<T>  
class Vector {  
    // ...  
};
```

Both versions are valid. The second version is nice because it makes the use of `requires` very explicit.

This improves our code in two ways:

- It documents the requirement clearly
- It gives much better compiler errors if someone tries to use a type that does not support equality

For example, this class would work fine:

```

class Point {
public:
    int x;
    int y;

    bool operator==(const Point& other) const {
        return x == other.x && y == other.y;
    }
};

```

So `Vector<Point>` would compile.

But this class would *not* satisfy the concept:

```

class Mystery {
public:
    int value;
};

```

Since `Mystery` does not define `operator==`, trying to make a `Vector<Mystery>` would fail at compile time in a much more readable way.

For this lab, go ahead and add the concept and use it to constrain your `Vector` template. A good final starting point would look like this:

```

#include <cstddef>
#include <stdexcept>
#include <concepts>

template <typename T>
concept EqualityComparable = requires(const T& a, const T& b) {
    { a == b } -> std::convertible_to<bool>;
};

template <EqualityComparable T>
class Vector {
    // ...
};

```

It's important to note also that this is a slight deviation from a normal vector. We've added a constraint that the types added to our `Vector` must be comparable, but no such constraint exists on a normal vector from the STL. Still, this is good practice.

3 Phase 3 Build an Iterator Step by Step

Now we are going to build an iterator for our vector.

An iterator is a nice generic wrapper around a pointer. In fact, the simplest iterator often just stores a pointer internally and then overloads a few operators so that it behaves like something you can move through.

In this phase, I will be much more explicit and walk you through the code.

Step 1 Start with a simple iterator class

Let us begin with a standalone iterator class. We will improve the design after that.

The iterator will store a pointer to the current element:

```
template <typename T>
class VectorIterator {
private:
    T* current_;

public:
    VectorIterator(T* ptr) : current_(ptr) {}
};
```

Here, `current_` is the pointer under the hood. That is what I mean when I say an iterator is often just a wrapper around a pointer.

Step 2 Overload operator*

We want to be able to dereference the iterator to get the current value.

Add this code:

```
T& operator*() {
    return *current_;
}
```

This means that if `it` is an iterator, then `*it` gives us the element it points to.

Step 3 Overload operator++

We also want to move the iterator forward.

Add this code:

```

VectorIterator& operator++() {
    ++current_;
    return *this;
}

```

This is pre-increment. It advances the iterator and then returns the iterator itself.

Step 4 Overload operator!=

To use a loop, we need to compare iterators.

Here is the function stub:

```
bool operator!=(const VectorIterator& other) const;
```

You should implement this yourself.

Think carefully:

- What does it mean for two iterators to be different?
- What data inside the iterator should be compared?

Step 5 Add begin() and end() to Vector

Now let us connect the iterator to the vector.

First, add these function declarations to your vector class:

```

VectorIterator<T> begin();
VectorIterator<T> end();

```

Then write the function bodies.

Remember:

- `begin()` should point at the first element
- `end()` should point one past the last element

So these should use:

- `data_`
- `data_ + size_`

Write both the declaration and the definition yourself.

Step 6 Use the iterator in a loop

Once your iterator, `begin()`, and `end()` are working, this kind of code should work:

```
Vector<int> v;
v.push_back(1);
v.push_back(2);
v.push_back(3);

for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << "\n";
}
```

Step 7 Improve the design by coupling the iterator to the vector

Right now the iterator is decoupled from the vector. It works, but it is nicer design-wise to make the iterator clearly belong to the vector class.

So instead of a separate class named `VectorIterator`, move it inside `Vector` as a nested class.

That makes it clear that this is a `Vector` iterator, not just some random iterator that could work for any data structure.

Here is the updated design:

```
template <typename T>
class Vector {
private:
    T* data_;
    size_t size_;
    size_t capacity_;

    void resize_capacity(size_t new_capacity);

public:
    class Iterator {
private:
        T* current_;

public:
        Iterator(T* ptr) : current_(ptr) {}
    };
};
```

```

    T& operator*() {
        return *current_;
    }

    Iterator& operator++() {
        ++current_;
        return *this;
    }

    bool operator!=(const Iterator& other) const;
};

Iterator begin();
Iterator end();
};

```

Using the scope resolution operator

When a class is nested inside another class, you refer to it using the scope resolution operator `::`.

For example:

```
Vector<int>::Iterator it = my_vector.begin();
```

That notation makes it very clear that this is specifically the iterator type for a `Vector<int>`.

Step 8 Make range-based for loops work

Once `begin()` and `end()` are implemented correctly, range-based for loops will work automatically.

For example:

```

Vector<std::string> words;
words.push_back("hello");
words.push_back("world");

for (const auto& word : words) {
    std::cout << word << "\n";
}

```

Under the hood, the compiler uses `begin()`, `end()`, `operator*()`, `operator++()`, and `operator!=()`.

So range-based for loops are just cleaner syntax on top of the iterator logic you built.

Const iterators

There is one more important issue.

Suppose the vector is `const`:

```
const Vector<int> nums;
```

Then calling `begin()` and `end()` as currently written will not work, because those are non-`const` member functions.

Just like before, we need `const`-aware versions.

For this lab, add these stubs and fill them in yourself:

```
class ConstIterator {
private:
    const T* current_;

public:
    ConstIterator(const T* ptr);

    const T& operator*() const;
    ConstIterator& operator++();
    bool operator!=(const ConstIterator& other) const;
};

ConstIterator begin() const;
ConstIterator end() const;
```

You should implement all of these.

Notice the differences:

- The pointer inside is now `const T*`
- Dereferencing returns `const T&`
- The `begin()` and `end()` methods themselves are now marked `const`

Recommended final public interface

By the end of this phase, your vector should roughly have a public interface like this:

```
template <typename T>
class Vector {
private:
    T* data_;
    size_t size_;
    size_t capacity_;

    void resize_capacity(size_t new_capacity);

public:
    class Iterator {
    private:
        T* current_;

    public:
        Iterator(T* ptr) : current_(ptr) {}
        T& operator*() { return *current_; }
        Iterator& operator++() {
            ++current_;
            return *this;
        }
        bool operator!=(const Iterator& other) const;
    };

    class ConstIterator {
    private:
        const T* current_;

    public:
        ConstIterator(const T* ptr);
        const T& operator*() const;
        ConstIterator& operator++();
        bool operator!=(const ConstIterator& other) const;
    };

    Vector();
```

```

~Vector();

size_t size() const;
size_t capacity() const;
bool empty() const;

void push_back(const T& value);

T& at(size_t index);
T& operator[](size_t index);
const T& operator[](size_t index) const;

bool operator==(const Vector<T>& other) const;

Iterator begin();
Iterator end();

ConstIterator begin() const;
ConstIterator end() const;
};

```

4 Wrapping Up Main Function Test Case

For this lab, the following `main()` function will serve as your test case.

If this compiles and runs correctly with your `vector.hpp` file, then your vector is working as expected for the features required in this lab.

```

#include <iostream>
#include <string>
#include "vector.hpp"

class Point {
public:
    int x;
    int y;

    Point(int x, int y) : x(x), y(y) {}

    bool operator==(const Point& other) const {

```

```

        return x == other.x && y == other.y;
    }
};

int main() {
    Vector<int> nums;
    std::cout << "Initially:\n";
    std::cout << "size = " << nums.size() << "\n";
    std::cout << "capacity = " << nums.capacity() << "\n";
    std::cout << "empty = " << nums.empty() << "\n\n";

    nums.push_back(10);
    nums.push_back(20);
    nums.push_back(30);

    std::cout << "After push_back:\n";
    std::cout << "size = " << nums.size() << "\n";
    std::cout << "capacity = " << nums.capacity() << "\n";
    std::cout << "nums.at(1) = " << nums.at(1) << "\n";
    std::cout << "nums[2] = " << nums[2] << "\n\n";

    int& ref = nums.at(1);
    ref = 999;

    std::cout << "After modifying reference from at():\n";
    std::cout << "nums.at(1) = " << nums.at(1) << "\n\n";

    Vector<int> other;
    other.push_back(10);
    other.push_back(999);
    other.push_back(30);

    std::cout << "nums == other: " << (nums == other) << "\n\n";

    std::cout << "Manual iterator loop:\n";
    for (auto it = nums.begin(); it != nums.end(); ++it) {
        std::cout << *it << " ";
    }
    std::cout << "\n\n";
}

```

```

std::cout << "Range-based for loop:\n";
for (const auto& value : nums) {
    std::cout << value << " ";
}
std::cout << "\n\n";

const Vector<int>& const_nums = nums;
std::cout << "Const range-based for loop:\n";
for (const auto& value : const_nums) {
    std::cout << value << " ";
}
std::cout << "\n\n";

Vector<std::string> words;
words.push_back("generic");
words.push_back("vector");
words.push_back("lab");

std::cout << "String vector:\n";
for (const auto& word : words) {
    std::cout << word << " ";
}
std::cout << "\n\n";

Vector<Point> points;
points.push_back(Point(1, 2));
points.push_back(Point(3, 4));

Vector<Point> points2;
points2.push_back(Point(1, 2));
points2.push_back(Point(3, 4));

std::cout << "points == points2: " << (points == points2) << "\n";

return 0;
}

```

Submission

Complete your `vector.hpp` file and verify that the provided `main.cpp` compiles and runs.

For this lab, that test program is your correctness check:

- If it compiles, your templates and interfaces are probably correct.
- If it runs and produces the expected behavior, then your vector implementation is working as expected.

As always, make sure your code is clean, readable, and properly commented where needed.